

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



SOICT

BÁO CÁO BÀI TẬP LỚN

XÂY DỰNG GAME DYNAMITE
TRÊN VI ĐIỀU KHIỂN STM32F429 DISCOVERY

Học phần: Hệ nhúng – IT4210

Giảng viên hướng dẫn: TS. Ngô Lam Trung

Nhóm sinh viên thực hiện: HAN Embedded

Trần Dương An – 20235256

Nguyễn Đào Nam Hải – 20235321

Đào Trọng Nguyên – 20235390

Hà Nội, tháng 7 năm 2026

MỤC LỤC

MỤC LỤC	2
1. GIỚI THIỆU	4
1.1 Mục tiêu đề tài	4
1.2 Nền tảng phần cứng và công cụ phát triển.....	4
1.3. Link mã nguồn project.....	4
2. PHÂN CÔNG CÔNG VIỆC.....	4
3. THIẾT KẾ PHẦN CỨNG	4
3.1 Sơ đồ khối tổng thể hệ thống	4
3.2 Bảng phân bổ chân (Pinout).....	5
3.3 Ngoại vi GPIO cho nút bấm.....	5
3.3.1 Cấu hình phần cứng (Hardware Configuration)	5
3.3.2 Cơ chế Polling — Lý do lựa chọn và cách triển khai	6
3.3.3 Phân loại hành vi xử lý – 3 kiểu khác nhau	7
3.3.4 Phân tích cơ chế Debounce	9
3.3.5 Bảng so sánh: Polling vs EXTI.....	9
3.4 Ngoại vi I2S kết nối MAX98357A.....	10
3.4.1 Ánh xạ ngoại vi và kết nối phần cứng	10
3.4.2 Cấu hình I2S3	10
3.4.3 Cơ chế DMA Circular – Kiến trúc Ping-Pong Buffer	11
3.4.4 Hàm fillAudioBuffer — Phân tích kỹ thuật.....	13
3.4.5 Nguyên lý hoạt động MAX98357A.....	14
3.4.6 Timing Diagram I2S — Chuẩn Philips, 16-bit, 16kHz	14
4. THIẾT KẾ PHẦN MỀM VÀ ÂM THANH	15
4.1 Định dạng và nén âm thanh	15
4.1.1 Lý thuyết và phân tích lựa chọn định dạng (PCM vs ADPCM).....	15
4.1.2 Danh sách âm thanh.....	15
4.1.3 Chi tiết cấu trúc dữ liệu AudioManager	16
4.2 Kiến trúc phần mềm TouchGFX (MVP)	16
4.2.1 Các thành phần chính.....	16
4.2.2 Phân tích luồng dữ liệu (Data Flow) trong MVP.....	17
4.2.3 Màn hình và Luồng màn hình.....	17
4.3 Thiết kế đối tượng Game (Game Objects) & Vật lý.....	18
4.3.1 Hệ thống tọa độ và Lưới lục giác (Hex Grid)	18
4.3.2 Cơ chế nhắm bắn và đạn bay	18

4.3.3 Thuật toán BFS xử lý nổ bóng và rút bóng.....	19
5. ĐÁNH GIÁ HIỆU NĂNG VÀ CÁC KHÓ KHĂN	19
5.1 Hiệu năng và Tài nguyên	19
5.1.1 Sử dụng bộ nhớ RAM.....	19
5.1.2 Sử dụng bộ nhớ Flash	20
5.1.3 Độ trễ xử lý input.....	20
5.2 Khó khăn trong project	20
5.2.1 Vấn đề 1: Lỗi Linker "Undefined reference"	20
5.2.2 Vấn đề 2: Nguy cơ tràn SRAM nội do Double Buffering	21
5.2.3 Vấn đề 3: Đánh đổi giữa giải nén ADPCM thời gian thực và tải CPU	21
5.2.4 Vấn đề 4: Thiếu cơ chế chống dội phím (debounce) phần cứng/phần mềm.....	21
6. KẾT LUẬN.....	22
6.1 Tổng kết kết quả đạt được.....	22
6.2 Hạn chế và hướng phát triển	22
6.3 Lời cảm ơn	23

1. GIỚI THIỆU

1.1 Mục tiêu đề tài

Xây dựng trò chơi Dynamite với khả năng xử lý đồ họa mượt mà (TouchGFX) và trộn âm thanh đa kênh (DMA I2S) trên vi điều khiển STM32F429I Discovery bị giới hạn tài nguyên.

1.2 Nền tảng phần cứng và công cụ phát triển

- Vi điều khiển: STM32F429ZIT6 (ARM Cortex-M4, 2MB Flash, 256KB RAM nội, 8MB SDRAM ngoại).
- Công cụ: STM32CubeIDE (sinh code HAL/C++), TouchGFX Designer 4.25.0.

1.3. Link mã nguồn project

<https://github.com/DuongAn15/DynamiteGame.git>

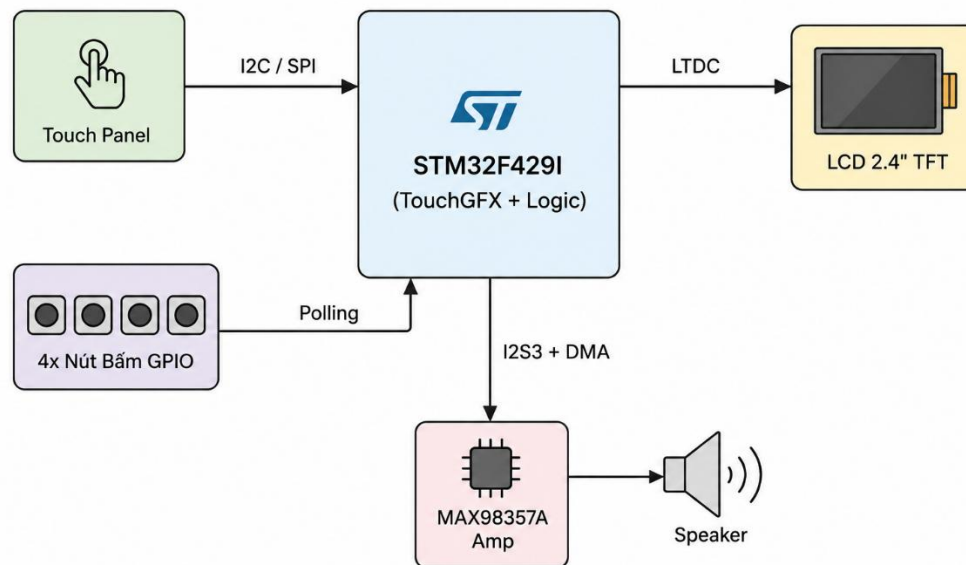
2. PHÂN CÔNG CÔNG VIỆC

Công việc chung: Thiết kế kiến trúc tổng thể, hỗ trợ nhau debug, kiểm thử tích hợp toàn hệ thống, biên soạn báo cáo.

Họ và Tên	MSSV	Nhiệm vụ thực hiện
Trần Dương An	20235256	Cấu hình phần cứng GPIO (4 nút bấm) và I2S/DMA; thiết kế giao diện và luồng chuyển màn hình.
Nguyễn Đào Nam Hải	20235321	Xây dựng hệ thống âm thanh (AudioManager): phát nhạc nền, hiệu ứng, giải mã ADPCM qua DMA I2S.
Đào Trọng Nguyên	20235390	Vẽ giao diện các màn hình trên TouchGFX Designer; xây dựng logic game (va chạm, gom nhóm bóng, phản xạ góc bắn).

3. THIẾT KẾ PHẦN CỨNG

3.1 Sơ đồ khối tổng thể hệ thống



3.2 Bảng phân bổ chân (Pinout)

Chức năng	Chân MCU	Chuẩn giao tiếp / Cấu hình
Đổi bóng (Swap)	PA5	GPIO Input, Pull-up
Bắn bóng (Shoot)	PA7	GPIO Input, Pull-up
Xoay trái (Left)	PG2	GPIO Input, Pull-up
Xoay phải (Right)	PG3	GPIO Input, Pull-up
I2S_DIN (Data)	PC12	I2S3_SD (Half_Duplex_Master)
I2S_BCLK (Clock)	PC10	I2S3_CK
I2S_LRC (WS)	PA15	I2S3_WS

3.3 Ngoại vi GPIO cho nút bấm

3.3.1 Cấu hình phần cứng (Hardware Configuration)

Project sử dụng 4 nút bấm vật lý nối trực tiếp vào các chân GPIO của MCU theo sơ đồ **Active-Low** (nhấn = kéo chân xuống GND). Bốn chân này được cấu hình thuần túy là *GPIO_Input*. Để đảm bảo chân GPIO có mức logic xác định khi nút không bấm, cả 4 chân đều được kích hoạt *điện trở kéo lên nội bộ (Internal Pull-up Resistor)* – không cần thêm linh kiện rời bên ngoài. Khi đó:

- **Nút không bấm:** Điện trở nội bộ kéo chân lên VDD → MCU đọc được mức logic HIGH (GPIO_PIN_SET).
- **Nút bấm:** Chân bị nối tắt xuống GND → MCU đọc được mức logic LOW (GPIO_PIN_RESET).

Cấu hình này được sinh tự động bởi STM32CubeIDE và ghi vào file .ioc

Configuration

Group By Peripherals

☒ GPIO
 ☒ FMC
 ☒ I2C
 ☒ I2S
 ☒ LTDC
 ☒ RCC
 ☒ SPI

Search Signals

Search (Ctrl+F) ☐ Show only Modified Pins

Pin Name	Signal on Pin	GPIO output ...	GPIO mode	GPIO Pull-up...	Maximum o...	User Label	Modified
PA5	n/a	n/a	Input mode	Pull-up	n/a		☒
PA7	n/a	n/a	Input mode	Pull-up	n/a		☒
PC1	n/a	High	Output Push...	No pull-up a...	Low	SPI5_NCS	☒
PC2	n/a	Low	Output Push...	No pull-up a...	High		☒
PD12	n/a	Low	Output Push...	No pull-up a...	High		☒
PD13	n/a	Low	Output Push...	No pull-up a...	High		☒
PE2	n/a	Low	Output Push...	No pull-up a...	Very High	VSYNC_FREQ	☒
PE3	n/a	Low	Output Push...	No pull-up a...	Very High	RENDER_TIME	☒
PE4	n/a	Low	Output Push...	No pull-up a...	Very High	FRAME_RATE	☒
PE5	n/a	Low	Output Push...	No pull-up a...	Very High	MCU_ACTIVE	☒
PG2	n/a	n/a	Input mode	Pull-up	n/a		☒

PA5 Configuration :

GPIO mode: Input mode

GPIO Pull-up/Pull-down: Pull-up

User Label:

3.3.2 Cơ chế Polling — Lý do lựa chọn và cách triển khai

Trong hệ thống nhúng, có hai phương pháp phổ biến để đọc trạng thái nút bấm:

1. **EXTI Interrupt:** MCU ngủ và bị đánh thức bởi tín hiệu sườn (rising/falling edge) từ chân GPIO — phản hồi tức thì, tiết kiệm CPU.
2. **Polling:** CPU chủ động đọc trạng thái thanh ghi GPIO theo chu kỳ – đơn giản, đồng bộ, không có vấn đề race condition.

Project này lựa chọn **Polling** vì lý do kiến trúc: TouchGFX hoạt động theo mô hình vòng lặp khung hình (*frame-based loop*) ở tần số **60 Hz** ($\approx 16.6 \text{ ms/frame}$).

Hàm *handleTickEvent()* của mỗi View được gọi một lần mỗi *frame* bởi framework. Đây là điểm tự nhiên nhất để đọc input — đọc và xử lý input trong cùng một ngữ cảnh thực thi, hoàn toàn tránh được vấn đề *race condition* giữa ISR và luồng chính.

Luồng thực thi Polling trong code:

Trích dẫn GameplayScreenView.cpp dòng 160-166 — phân đọc GPIO:

```

// =====
// HARDWARE BUTTON LOGIC (Direct Polling)
// =====
bool isLeft  = (HAL_GPIO_ReadPin(GPIOD, GPIO_PIN_2) == GPIO_PIN_RESET);
bool isRight = (HAL_GPIO_ReadPin(GPIOD, GPIO_PIN_3) == GPIO_PIN_RESET);
bool isSwap  = (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_5) == GPIO_PIN_RESET);
bool isShoot = (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_7) == GPIO_PIN_RESET);
  
```

Vì logic Active-Low, biểu thức `== GPIO_PIN_RESET` trả về true khi nút đang được nhấn – kết quả được lưu vào biến bool cục bộ.

3.3.3 Phân loại hành vi xử lý – 3 kiểu khác nhau

Sau khi đọc trạng thái GPIO, project **KHÔNG** xử lý cả 4 nút theo cùng một cách. Tùy theo ngữ nghĩa gameplay, mỗi nhóm nút có cơ chế xử lý riêng:

a) Kiểu 1 – Giữ liên tục (Continuous Hold): nút Left, Right

Nút Left và Right dùng để di chuyển góc ngắm. Hành vi mong muốn là: **giữ nút → góc ngắm dịch chuyển liên tục** mỗi frame. Do đó, code kiểm tra trạng thái trực tiếp (không cần so sánh với frame trước):

Trích dẫn GameplayScreenView.cpp dòng 168-184:

```

if (isLeft || isRight) {
    if (!isAiming) {
        isAiming = true;
        aimX = (int)smoothedAimX;
        aimY = BULLET_START_Y - AIM_START_Y_OFFSET; // = 243 - 50 = 193 pixel
    }

    if (isLeft) {
        aimX -= 1;
        if (aimX < 0) aimX = 0; // Chặn biên trái màn hình
    } else if (isRight) {
        aimX += 1;
        if (aimX > SCREEN_WIDTH) aimX = SCREEN_WIDTH; // Chặn biên phải (240px)
    }

    presenter->handleTouchAim(aimX, aimY);
}
  
```

Mỗi frame khi giữ nút, *aimX* dịch chuyển ± 1 pixel. Ở 60 Hz, điều này cho tốc độ di chuyển **60 pixel/giây** – vừa đủ chính xác cho gameplay. Biên được chặn cứng trong đoạn $[0, SCREEN_WIDTH]$.

b) Kiểu 2 — Phát hiện sườn lên (Rising Edge Detection): nút Swap

Nút *Swap* đổi màu quả bóng hiện tại và quả dự bị. Hành vi mong muốn: *mỗi lần nhấn chỉ thực hiện một lần đổi*, dù giữ bao lâu. Đây là bài toán *phát hiện sườn lên* (rising edge – trạng thái chuyển từ *low* $\rightarrow$ *high*).

Giải pháp: dùng biến *prevSwap* lưu trạng thái frame trước. Chỉ kích hoạt lệnh đổi màu khi *isSwap* == *true* VÀ *prevSwap* == *false*:

Trích dẫn *GameplayScreenView.cpp* dòng 186-189:

```
if (isSwap && !prevSwap) {           // Điều kiện: frame này nhấn, frame trước chưa nhấn
    presenter->handleSwapColor(); // Thực hiện đổi màu đúng 1 lần
}
prevSwap = isSwap;                   // Lưu trạng thái hiện tại cho frame kế tiếp
```

c) Kiểu 3 — Phát hiện sườn lên (Rising Edge Detection): nút Shoot

Nút *Shoot* bắn bóng. Tương tự *Swap*, mỗi lần nhấn chỉ bắn một viên. Ngoài ra, logic bắn có hai trường hợp phụ thuộc vào cờ *isAiming*:

Trích dẫn *GameplayScreenView.cpp* dòng 191-199:

```
if (isShoot && !prevShoot) {           // Rising edge: chỉ kích hoạt lần đầu nhấn
    if (isAiming) {
        isAiming = false;
        // Bắn theo hướng đã ngắm
        presenter->handleTouchShoot((int)smoothedAimX, (int)smoothedAimY);
    } else {
        // Bắn thẳng lên (x=120, y=0)
        presenter->handleTouchShoot(BULLET_START_X, 0);
    }
}
prevShoot = isShoot;
```

- Nếu *isAiming* == *true*: người chơi đã ngắm từ trước (qua touch hoặc Left/Right) $\rightarrow$ bắn theo hướng *smoothedAimX/Y*.
- Nếu *isAiming* == *false*: chưa ngắm $\rightarrow$ bắn thẳng lên (mặc định *y* = 0).

Lưu ý quan trọng về khởi tạo *prevShoot* và *prevSwap*:

Hai biến này được khởi tạo đồng bộ với trạng thái phần cứng thực tế ngay tại *setupScreen()*, tránh trường hợp nút đang giữ sẵn khi chuyển màn hình bị nhận nhầm là "vừa nhấn":

Trích dẫn *GameplayScreenView.cpp* dòng 31-32:

```
prevShoot = (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_7) == GPIO_PIN_RESET);
prevSwap = (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_5) == GPIO_PIN_RESET);
```

Cơ chế tương tự cũng được áp dụng ở *HomeScreenView* – màn hình Menu cũng poll nút *Shoot* để chuyển sang màn chơi:

Trích dẫn *HomeScreenView.cpp* dòng 26-31:


```
bool isShoot = (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_7) == GPIO_PIN_RESET);
if (isShoot && !prevShoot) {
    application().gotoGameplayScreenScreenCoverTransitionEast();
}
prevShoot = isShoot;
```

3.3.4 Phân tích cơ chế Debounce

a) Debounce là gì và tại sao cần thiết?

Khi nút bấm cơ học chuyển trạng thái, tiếp điểm kim loại không đóng/mở dứt khoát ngay lập tức mà *dao động (bounce)* trong khoảng 1–20 ms trước khi ổn định. Nếu không lọc, MCU có thể đọc nhầm nhiều lần nhấn từ một lần bấm vật lý duy nhất.

b) Debounce trong project này:

Project *KHÔNG* triển khai thuật toán debounce chuyên dụng theo thời gian (ví dụ: đếm N frame liên tiếp cùng trạng thái, hoặc delay T ms qua SysTick). Cơ chế phát hiện sườn (prevShoot/prevSwap) *không phải là debounce* – nó chỉ đảm bảo lệnh thực thi đúng 1 lần mỗi lần nhấn, nhưng nếu bounce xảy ra trong đúng khoảng cách 2 frame (≈ 16.6 ms), nó vẫn có thể tạo ra 2 sự kiện.

Tuy nhiên, điều này được *giảm thiểu tự nhiên* bởi chu kỳ Polling 60 Hz (≈ 16.6 ms/frame). Vì bounce thường kết thúc trong vòng 5–10 ms, và frame tick cách nhau 16.6 ms, xác suất một bounce kịp tạo ra 2 rising edge qua 2 frame liên tiếp là thấp – *hệ thống hoạt động ổn định trong thực tế*, dù về lý thuyết vẫn có khả năng xảy ra lỗi debounce với các nút có chất lượng tiếp điểm kém.

3.3.5 Bảng so sánh: Polling vs EXTI

Bảng 1 – So sánh phương án đọc nút bấm

Tiêu chí	Polling (đang dùng)	EXTI Interrupt (phương án thay thế)
Cơ chế	CPU đọc thanh ghi GPIO chủ động mỗi 16.6 ms	MCU bị đánh thức bởi sườn tín hiệu từ GPIO
Độ trễ phản hồi	Tối đa 16.6 ms (1 frame)	Tức thì ($< 1 \mu s$ từ lúc nhấn đến ISR)
Tải CPU	~ 4 lần đọc thanh ghi/frame – không đáng kể	Gần bằng 0 khi không bấm
Độ phức tạp code	Rất thấp – tuần tự, cùng luồng với UI	Cao – cần biến volatile, flag, ISR tách biệt
Nguy cơ Race condition	Không có – đọc và dùng trong cùng 1 luồng	Có – ISR có thể chen ngang giữa chừng vòng lặp chính
Debounce	Tự nhiên qua chu kỳ 16.6 ms	Bắt buộc phải implement tường minh trong ISR

Tiêu chí	Polling (đang dùng)	EXTI Interrupt (phương án thay thế)
Phù hợp với TouchGFX	Rất phù hợp – đồng bộ với vòng lặp frame	Cần cẩn thận khi cập nhật UI từ ISR context

3.4 Ngoại vi I2S kết nối MAX98357A

3.4.1 Ảnh xạ ngoại vi và kết nối phần cứng

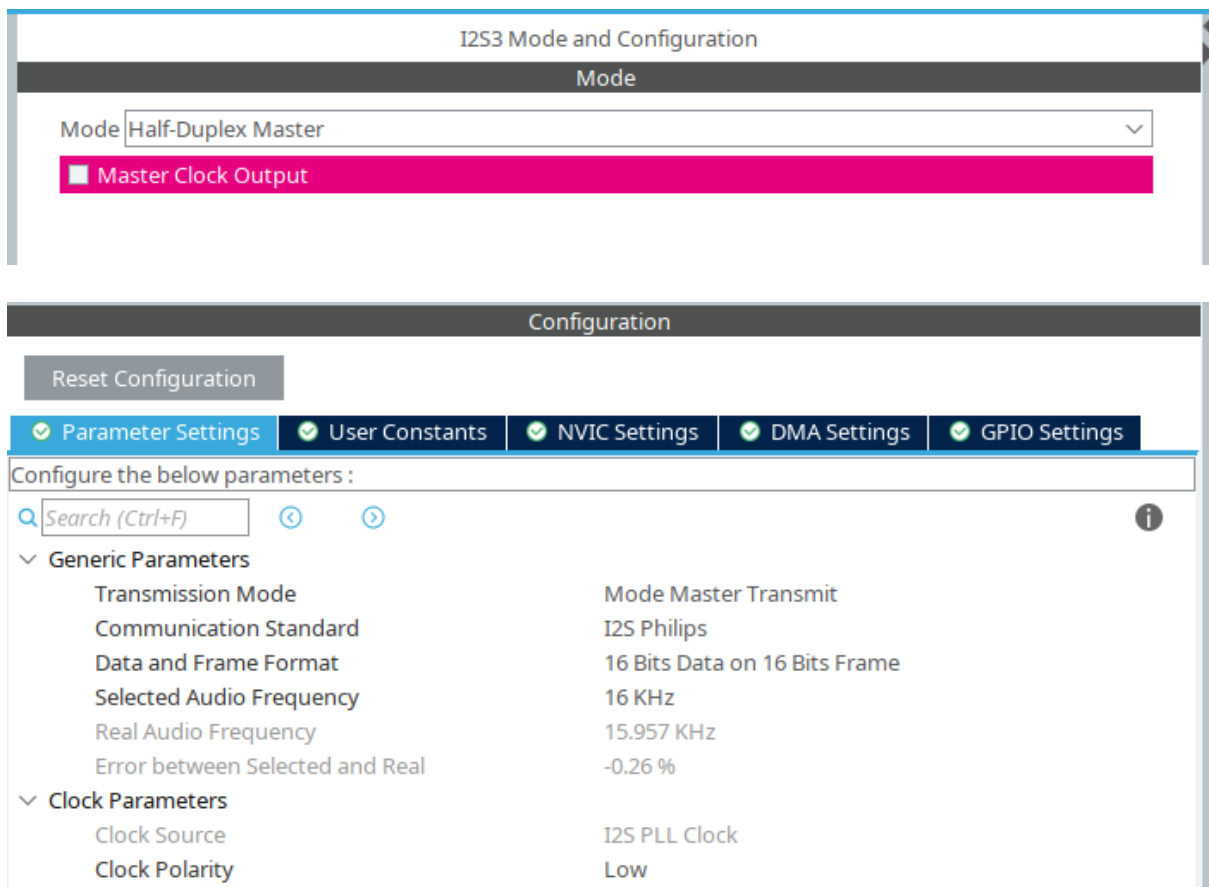
STM32F429 không có ngoại vi I2S độc lập – thay vào đó, nó **tái sử dụng bộ SPI3 và cấu hình lại thành giao thức I2S** thông qua cơ chế remap nội bộ.

Giao thức I2S yêu cầu tối thiểu **3 đường tín hiệu**, được gán tại các chân cụ thể:

Tín hiệu I2S	Tên chân STM32	Alternate Function	Vai trò
WS (Word Select / LRC)	PA15	I2S3_WS	Phân biệt kênh Trái/Phải (0=Left, 1=Right)
CK (Bit Clock / BCLK)	PC10	I2S3_CK	Xung nhịp truyền từng bit
SD (Serial Data / DIN)	PC12	I2S3_SD	Luồng dữ liệu từ MCU tới MAX98357A

Không sử dụng MCLK (Master Clock): MAX98357A là bộ khuếch đại Class-D tích hợp ADC kỹ thuật số, không cần đường MCLK riêng để hoạt động – nó tự đồng bộ nhịp từ tín hiệu BCLK và WS.

3.4.2 Cấu hình I2S3



I2S3 Mode and Configuration

Mode

Mode: Half-Duplex Master

☒ Master Clock Output

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings

Configure the below parameters :

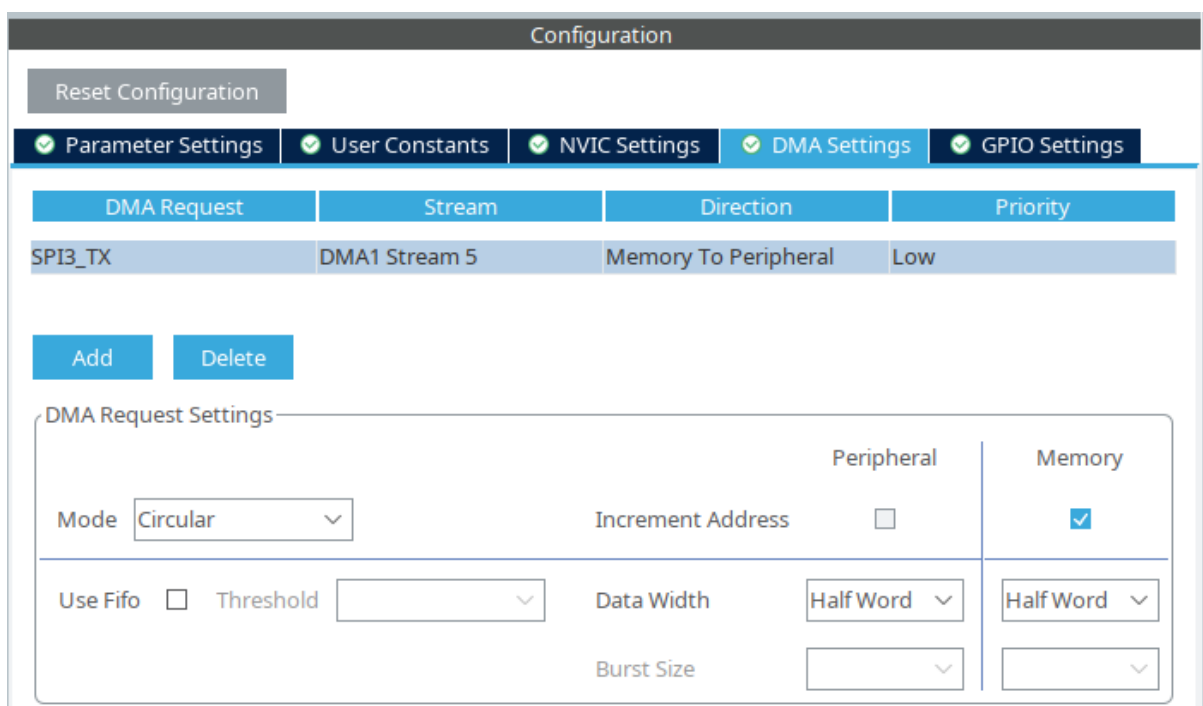
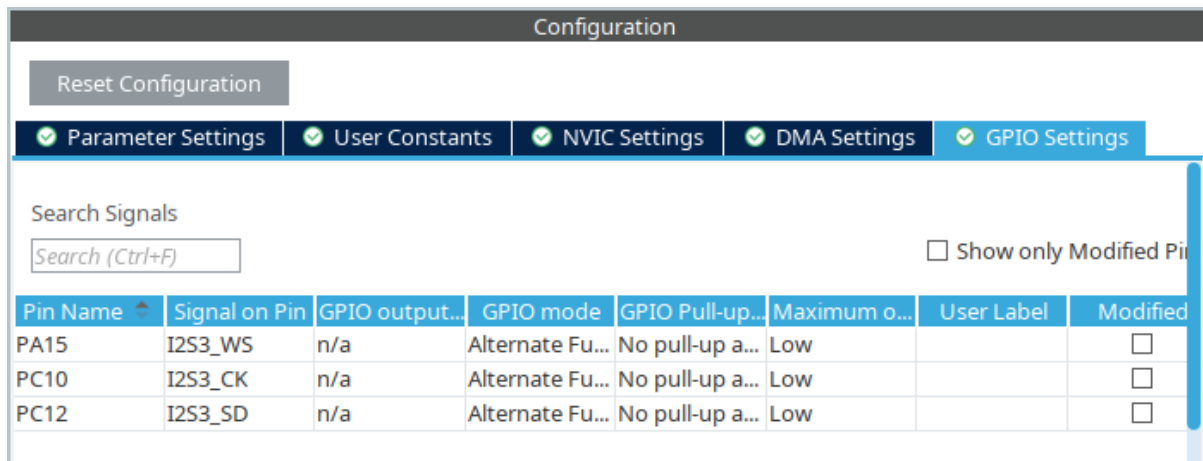
Search (Ctrl+F)

Generic Parameters

Transmission Mode	Mode Master Transmit
Communication Standard	I2S Philips
Data and Frame Format	16 Bits Data on 16 Bits Frame
Selected Audio Frequency	16 KHz
Real Audio Frequency	15.957 KHz
Error between Selected and Real	-0.26 %

Clock Parameters

Clock Source	I2S PLL Clock
Clock Polarity	Low



3.4.3 Cơ chế DMA Circular – Kiến trúc Ping-Pong Buffer

a) Định nghĩa buffer:

Trích dẫn *AudioManager.cpp* dòng 16-17:

```
#define AUDIO_BUFFER_SIZE 4096
static uint16_t audio_buffer[AUDIO_BUFFER_SIZE];
```

audio_buffer là mảng **4096 phần tử uint16_t = 8192 byte** tổng, được khai báo *static* – nằm cố định trong SRAM nội của MCU trong suốt vòng đời chương trình.

b) Khởi động DMA:

Trích dẫn *AudioManager.cpp* dòng 88-95:

```
void AudioManager::init() {
    if (!initialized) {
        for (int i = 0; i < AUDIO_BUFFER_SIZE; ++i) {
            audio_buffer[i] = 0; // Xóa buffer về 0 (im lặng)
        }
        HAL_I2S_Transmit_DMA(&hi2s3, (uint16_t*)audio_buffer, AUDIO_BUFFER_SIZE);
        initialized = true;
    }
}
```

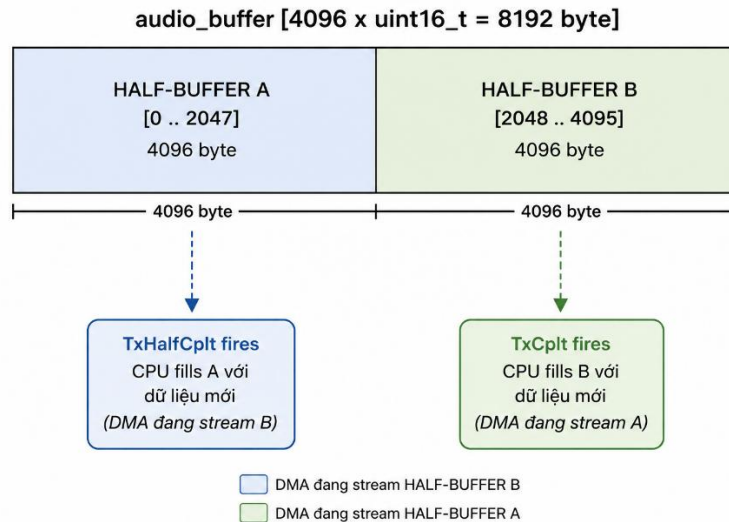
```
}

```

HAL_I2S_Transmit_DMA() khởi động DMA với toàn bộ 4096 phần tử. DMA được cấu hình *DMA_CIRCULAR* – sau khi truyền hết 4096 phần tử, DMA tự động quay về đầu buffer và lặp lại vô tận, *không cần MCU can thiệp restart*.

c) Cơ chế Ping-Pong (Double Buffering):

Kỹ thuật này chia *audio_buffer* thành 2 nửa bằng nhau (mỗi nửa 2048 phần tử = 4096 byte). DMA stream liên tục theo chiều mũi tên, CPU được thông báo để nạp dữ liệu mới vào nửa vừa được stream xong:



Trích dẫn *AudioManager.cpp* dòng 191-201 — 2 DMA callback:

```
extern "C" void HAL_I2S_TxHalfCpltCallback(I2S_HandleTypeDef *hi2s) {
    if (hi2s->Instance == SPI3) {
        // Fill nửa A
        fillAudioBuffer(&audio_buffer[0], AUDIO_BUFFER_SIZE / 2);
    }
}

extern "C" void HAL_I2S_TxCpltCallback(I2S_HandleTypeDef *hi2s) {
    if (hi2s->Instance == SPI3) {
        // Fill nửa B
        fillAudioBuffer(&audio_buffer[AUDIO_BUFFER_SIZE / 2], AUDIO_BUFFER_SIZE / 2);
    }
}
```

Phân tích thời gian (Latency):

Ở tần số thực tế 15,957 Hz, mỗi half-buffer chứa **2048 sample**:

- Thời gian DMA stream 1 half-buffer: $2048 / 15,957 \approx 128$ ms
- CPU có **128 ms** để decode ADPCM và fill 2048 sample trước khi DMA quay lại nửa đó
- Đây là khoảng thời gian rất thoải mái cho Cortex-M4 (180 MHz)

Lợi ích khi sử dụng kiến trúc này: CPU và DMA hoạt động hoàn toàn song song – trong khi DMA stream nửa B ra speaker, CPU decode và fill nửa A. Không có thời điểm nào CPU phải chờ DMA hoặc ngược lại.

3.4.4 Hàm *fillAudioBuffer* — Phân tích kỹ thuật

fillAudioBuffer() là trái tim của hệ thống âm thanh, thực hiện *decode* + *mix* + *ghi buffer* trong một lần duyệt:

Trích dẫn *AudioManager.cpp* dòng 140-188:

```
static void fillAudioBuffer(uint16_t* buffer, uint32_t size) {
    for (uint32_t i = 0; i < size; i += 2) { // Bước nhảy 2: mỗi lần fill 1 cặp L+R
        int32_t mixed_sample = 0; // Accumulator 32-bit tránh overflow

        // [1] Decode ADPCM từ BGM channel
        if (bgmChannel.active && bgmChannel.data) {
            uint8_t byte = bgmChannel.data[bgmChannel.position];
            uint8_t nibble = bgmChannel.is_nibble_high ? (byte >> 4) : (byte & 0x0F);
            int16_t sample = adpcm_decode(nibble, bgmChannel.predictor,
            bgmChannel.step_index);
            mixed_sample += sample;
            // ... advance position, handle looping ...
        }

        // [2] Mix tối đa 3 kênh SFX (PCM 16-bit trực tiếp)
        for (int ch = 0; ch < 3; ++ch) {
            if (sfxChannels[ch].active && sfxChannels[ch].data) {
                int16_t sample =
                (int16_t)sfxChannels[ch].data[sfxChannels[ch].position];
                mixed_sample += sample;
                // ... advance position ...
            }
        }

        // [3] Clamp kết quả về [-32768, 32767] (tránh wraparound khi mix nhiều kênh)
        if (mixed_sample > 32767) mixed_sample = 32767;
        else if (mixed_sample < -32768) mixed_sample = -32768;

        // [4] Ghi cùng 1 sample vào CẢ HAI vị trí L và R – tạo MONO qua I2S Stereo
        buffer[i] = (uint16_t)(int16_t)mixed_sample; // Kênh Trái (L)
        if (i + 1 < size)
            buffer[i + 1] = (uint16_t)(int16_t)mixed_sample; // Kênh Phải (R) = giống L
    }
}
```

Điểm quan trọng – Chiến lược Mono qua I2S Stereo (dòng 184-187):

Chuẩn I2S luôn truyền theo frame stereo (L + R). Tuy nhiên, project chỉ có 1 kênh *output mono* (loa đơn qua MAX98357A). Giải pháp là ghi *cùng một giá trị* *mixed_sample* vào cả *buffer[i]* (kênh Trái) lẫn *buffer[i+1]* (kênh Phải). Kết quả là MAX98357A nhận được L = R, tính $(L+R)/2 = mixed_sample$ – output mono hoàn hảo.

Cơ chế decode ADPCM on-the-fly:

Mỗi byte ADPCM lưu 2 nibble 4-bit (2 sample). Biến *is_nibble_high* theo dõi nibble nào đang được đọc: *true* = đọc 4 bit cao (*byte >> 4*), *false* = đọc 4 bit thấp (*byte & 0x0F*). Position chỉ tăng sau khi đọc xong nibble thấp. Cặp trạng thái {predictor, step_index} được duy trì liên tục giữa các lần gọi callback – đảm bảo giải mã ADPCM liền mạch qua ranh giới half-buffer.

3.4.5 Nguyên lý hoạt động MAX98357A

Theo datasheet MAX98357A (Maxim Integrated), chip này là **bộ khuếch đại Class-D stereo-to-mono** tích hợp bộ giải mã I2S. Nguyên lý hoạt động:

1. **Nhận dữ liệu I2S stereo:** Chip tự đồng bộ từ BCLK và WS – không cần MCLK ngoài.
2. **Mix cứng $(L+R)/2$:** Chip luôn tự động tính trung bình cộng hai kênh rồi đưa ra ngõ ra mono duy nhất. Đây là tính năng cố định trong phần cứng của chip – phần mềm không thể can thiệp.
3. **Chân SD (Shutdown/Gain Select):** Không liên quan đến chọn kênh. Chức năng duy nhất:
 - Kéo về GND → Chip tắt hoàn toàn (Shutdown)
 - Để thả nổi / phân áp → Chọn mức gain: 6 dB / 9 dB / 12 dB / 15 dB tùy theo điện áp tại chân

Project này thiết lập $L = R$ như phân tích ở mục 3.4.4, giá trị sau khi MAX98357A mix sẽ là $(L + R) / 2 = (mixed_sample + mixed_sample) / 2 = mixed_sample$ – đầu ra loa rất giống với dữ liệu phần mềm tạo ra.

3.4.6 Timing Diagram I2S — Chuẩn Philips, 16-bit, 16kHz

Thông số tín hiệu:

- BCLK (PC10): $15,957\text{ Hz} \times 16\text{ bit/sample} \times 2\text{ channel} = 510,624\text{ Hz} \approx 511\text{ kHz}$ (thực tế, do sai lệch PLL -0.26%)
- WS (PA15): $15,957\text{ Hz}$ – chu kỳ 1 frame = $\sim 62.7\text{ }\mu\text{s}$
- DIN (PC12): Dữ liệu PCM 16-bit, MSB trước, kênh Trái rồi kênh Phải

Thời điểm (chu kỳ BCLK)	WS / LRC (PA15)	DIN (PC12)	Ý nghĩa
Chu kỳ 0	Low → (chuyển trạng thái)	LSB (Bit 0) của frame trước	Trễ chuẩn Philips — WS vừa xuống Low nhưng DIN vẫn đang truyền bit cuối của frame trước
Chu kỳ 1	Low (Kênh Trái)	Bit 15 (MSB) – Kênh L	Bắt đầu truyền sample Trái (trễ 1 chu kỳ so với WS)
Chu kỳ 2–15	Low (Kênh Trái)	Bit 14 → Bit 1	Lần lượt các bit tiếp theo của sample L
Chu kỳ 16	High → (chuyển trạng thái)	Bit 0 (LSB) – Kênh L	Truyền bit cuối Kênh Trái. Đồng thời , WS chuyển lên High sớm 1 chu kỳ để chuẩn bị cho Kênh Phải
Chu kỳ 17	High (Kênh Phải)	Bit 15 (MSB) – Kênh R	Bắt đầu truyền sample Phải
Chu kỳ 18–31	High (Kênh Phải)	Bit 14 → Bit 1	Lần lượt các bit tiếp theo của sample R
Chu kỳ 32 (0)	Low → (chuyển trạng thái)	Bit 0 (LSB) – Kênh R	Truyền bit cuối Kênh Phải. WS lại chuyển xuống Low để lặp lại chu kỳ mới

Như đã phân tích tại mục 3.4.4, $buffer[i] = buffer[i+1] = mixed_sample$. Do vậy, giá trị bit trên đường DIN ở kênh Trái và kênh Phải *hoàn toàn giống nhau* trong mọi frame.

4. THIẾT KẾ PHẦN MỀM VÀ ÂM THANH

4.1 Định dạng và nén âm thanh

4.1.1 Lý thuyết và phân tích lựa chọn định dạng (PCM vs ADPCM)

Trong các hệ thống nhúng với tài nguyên bộ nhớ hạn hẹp như vi điều khiển STM32F429 (chỉ có 2MB Flash nội bộ), việc lưu trữ âm thanh chất lượng cao là một thách thức lớn. Tốc độ lấy mẫu 16kHz ở 16-bit tốn khoảng 32 KB mỗi giây. Để giải quyết bài toán này, chiến lược lưu trữ kết hợp giữa hai định dạng **PCM** và **ADPCM** đã được áp dụng nhằm cân bằng giữa dung lượng bộ nhớ và hiệu năng CPU.

a) Định dạng PCM (Pulse-Code Modulation) – Dành cho hiệu ứng âm thanh (SFX)

- **Nguyên lý:** PCM là định dạng âm thanh thô (uncompressed). Mỗi mẫu âm thanh (sample) được lấy mẫu và lưu trữ trực tiếp dưới dạng một số nguyên tuyệt đối. Trong project, các file SFX dùng chuẩn PCM 16-bit (mỗi sample chiếm 2 byte).
- **Phân tích lựa chọn:** Các âm thanh hiệu ứng (như tiếng click nút bấm, bắn bóng, tiếng bóng vỡ) có thời lượng rất ngắn, thường chỉ là một phần nhỏ của giây.
 - Việc lưu bằng PCM tốn thêm chút ít dung lượng nhưng **đổi lại CPU không mất thời gian giải mã**.
 - Điều này giúp hệ thống đáp ứng tức thì (low latency) và tiết kiệm tài xử lý khi phải mix nhiều kênh âm thanh hiệu ứng (SFX channels) cùng một lúc.

b) Định dạng ADPCM (Adaptive Differential Pulse-Code Modulation) – Dành cho nhạc nền (BGM)

- **Nguyên lý:** Khác với PCM lưu giá trị tuyệt đối, ADPCM là thuật toán nén âm thanh lưu trữ **sự chênh lệch (difference)** giữa mẫu hiện tại và mẫu dự đoán trước đó. Bảng bước nhảy (step table) thay đổi linh hoạt (adaptive) để bám theo độ dốc của sóng âm.
- **Hiệu quả nén:** Thuật toán ADPCM biến đổi một sample 16-bit gốc thành một mã 4-bit (1 nibble). Nhờ đó, ADPCM đạt **tỷ lệ nén 4:1** so với PCM 16-bit. Lấy ví dụ: mảng nhạc nền 512,000 byte thực chất chứa 1,024,000 sample, tương đương 64 giây âm thanh.
- **Phân tích lựa chọn:** Các bản nhạc nền kéo dài hơn 1 phút. Nếu dùng định dạng PCM 16-bit, 1 phút nhạc sẽ tốn gần 2 MB, lập tức làm tràn bộ nhớ Flash của MCU. Áp dụng ADPCM giúp giảm dung lượng file âm thanh xuống 4 lần, cho phép toàn bộ game "nằm lọt thỏm" vào 2MB Flash nội bộ mà không cần sử dụng thêm thẻ nhớ SD hay chip Flash SPI ngoại vi. Đổi lại, MCU phải tốn thêm một chút tài tính toán để giải mã, nhưng chip Cortex-M4 (180MHz) hoàn toàn dư sức thực hiện việc này.

4.1.2 Danh sách âm thanh

Tên âm thanh	Loại	Định dạng	Kích thước mảng chính xác
audio_bgm_menu	Nhạc nền (BGM)	ADPCM (uint8_t)	512.000 byte (~500 KB)
audio_game_play	Nhạc nền (BGM)	ADPCM (uint8_t)	496.000 byte (~484 KB)
audio_sfx_gameover	Hiệu ứng (SFX)	PCM 16-bit (uint16_t)	$64.000 \times 2 = 128.000$ byte
audio_sfx_drop	Hiệu ứng (SFX)	PCM 16-bit (uint16_t)	$9.600 \times 2 = 19.200$ byte

Tên âm thanh	Loại	Định dạng	Kích thước mảng chính xác
audio_sfx_match	Hiệu ứng (SFX)	PCM 16-bit (uint16_t)	$6.400 \times 2 = 12.800$ byte
audio_sfx_button	Hiệu ứng (SFX)	PCM 16-bit (uint16_t)	$2.400 \times 2 = 4.800$ byte
audio_sfx_shoot	Hiệu ứng (SFX)	PCM 16-bit (uint16_t)	$2.400 \times 2 = 4.800$ byte
audio_sfx_hit	Hiệu ứng (SFX)	PCM 16-bit (uint16_t)	$3.200 \times 2 = 6.400$ byte

4.1.3 Chi tiết cấu trúc dữ liệu AudioManager

Project không sử dụng enum SoundID, mà thiết kế linh hoạt bằng cách truyền trực tiếp con trỏ mảng byte vào hàm phát.

Ví dụ: `AudioManager::playBGM(audio_game_play, audio_game_play_length)`

Dữ liệu trạng thái của các kênh âm thanh phản ánh rõ bản chất định dạng của chúng và được định nghĩa trong `AudioManager.cpp`.

Trích dẫn `AudioManager.cpp` dòng 19-36:

```
// Kênh BGM chứa các biến trạng thái phục vụ giải mã ADPCM
struct BgmChannel {
    const uint8_t* data;    // Con trỏ mảng uint8_t (ADPCM 4-bit)
    uint32_t length;
    uint32_t position;      // Vị trí byte đang đọc
    bool active;
    bool looping;
    int16_t predictor;      // [ADPCM] Bộ dự đoán mẫu hiện tại
    int8_t step_index;      // [ADPCM] Chỉ số bước nhảy trong step_table
    bool is_nibble_high;    // [ADPCM] Cờ xác định đang đọc nửa 4-bit cao hay thấp
};

// Kênh SFX đơn giản hơn do chỉ cần truyền PCM thô
struct SfxChannel {
    const uint16_t* data;   // Con trỏ mảng uint16_t (PCM 16-bit)
    uint32_t length;
    uint32_t position;
    bool active;
    bool looping;
};
```

4.2 Kiến trúc phần mềm TouchGFX (MVP)

4.2.1 Các thành phần chính

Thư mục Screen	View class	Presenter class	Nhiệm vụ chính
homescreen_screen	HomeScreenView	HomeScreenPresenter	Màn hình Menu (Hiển thị nút Play, xử lý poll nút Shoot).

Thư mục Screen	View class	Presenter class	Nhiệm vụ chính
gameplayscreen_screen	GameplayScreenView	GameplayScreenPresenter	Vòng lặp game chính (Vẽ lưới trứng, đạn bay, nòng súng).
endscreen_screen	EndScreenView	EndScreenPresenter	Màn hình Pause / Game Over

4.2.2 Phân tích luồng dữ liệu (Data Flow) trong MVP

Tất cả các Presenter đều chứa một con trỏ trỏ đến *cùng một instance Model duy nhất*. Sự kiện diễn ra theo chu trình kín:

- **(1) Input (Từ View → Presenter → Model):**

Khi người chơi thao tác (VD: click chuột, nhấn nút GPIO), GameplayScreenView gọi hàm presenter->handleTouchShoot(). Presenter chỉ đóng vai trò trung chuyển (mediator), gọi tiếp model->handleTouchShoot().

Toàn bộ logic tính toán vector vận tốc (vx, vy) diễn ra tại Model.

- **(2) Cập nhật trạng thái (Trong Model):**

Model::tick() chạy liên tục mỗi frame để cập nhật tọa độ đạn (updateFlyingPhysics), kiểm tra va chạm, ăn điểm.

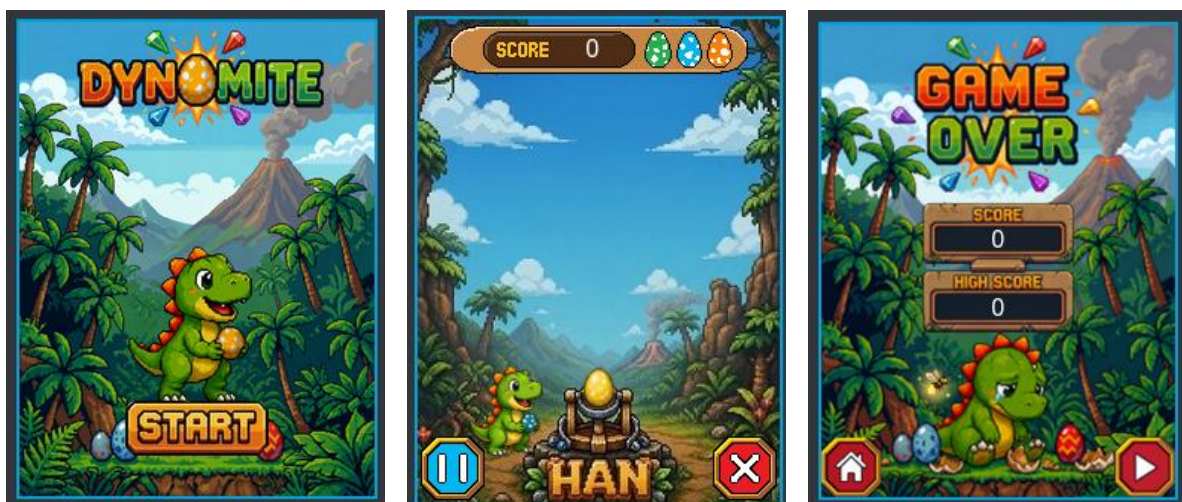
- **(3) Output (Từ Model → Presenter → View):**

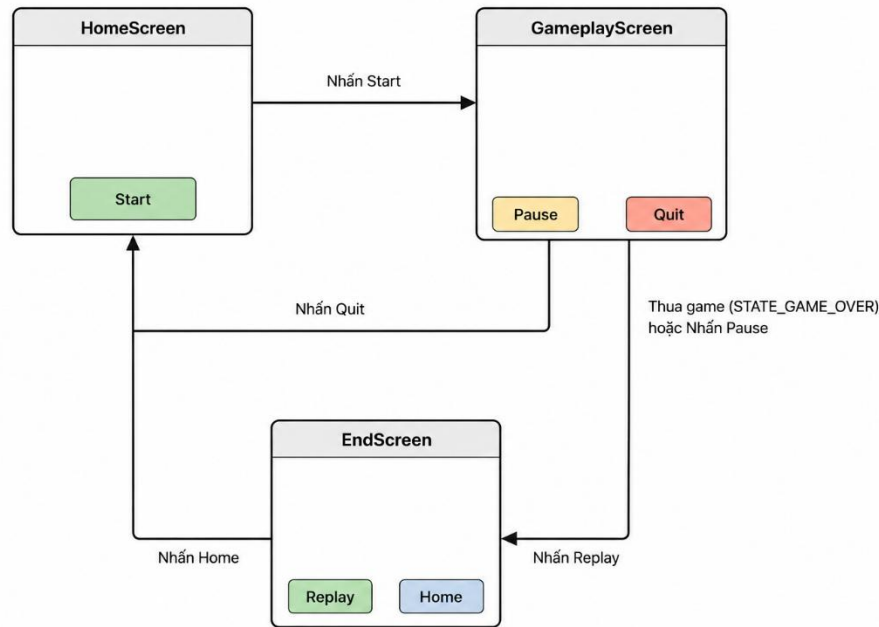
Khi có sự kiện thay đổi (VD: điểm số tăng), Model không gọi View trực tiếp mà thông qua interface ModelListener.

Model gọi modelListener->notifyScoreUpdated(score). GameplayScreenPresenter (kế thừa ModelListener) nhận tín hiệu này và gọi ngược lại view.updateScore(score).

Nhờ thiết kế này, logic game (Model.cpp) hoàn toàn không phụ thuộc vào thư viện đồ họa TouchGFX, dễ dàng bảo trì hoặc port sang một nền tảng đồ họa khác nếu cần.

4.2.3 Màn hình và Luồng màn hình





4.3 Thiết kế đối tượng Game (Game Objects) & Vật lý

Toàn bộ logic tính toán vật lý và lưới trò chơi được cô lập trong `Model.cpp` và `GameConstants.hpp`.

4.3.1 Hệ thống tọa độ và Lưới lục giác (Hex Grid)

Trò chơi không sử dụng lưới vuông mà dùng lưới tổ ong (Hex Grid) để các quả bóng xếp đan xen vào nhau một cách chặt chẽ.

- Mảng logic: mảng 1 chiều `uint8_t grid[MAX_ROWS * MAX_COLS]` ($10 \times 9 = 90$ ô).
- Chuyển đổi tọa độ: Hàng chẵn và hàng lẻ có tọa độ trục X lệch nhau đúng nửa đường kính quả bóng ($0.5 * CELL_WIDTH = 12 \text{ pixel}$).

Trích dẫn `Model.cpp` dòng 404-408:

```
void Model::getCellCenter(int col, int row, float &px, float &py) const
{
    px = ((row + gridParityOffset) & 1) == 0 ? MODEL_CELL_X_EVEN[col] :
MODEL_CELL_X_ODD[col];
    py = MODEL_CELL_Y[row] + globalOffsetY;
}
```

Nhờ ma trận lookup lân cận `NEIGHBOR_OFFSETS[2][6][2]` (chứa sẵn offset (dx, dy) và phân biệt chẵn/lẻ), thuật toán có thể dễ dàng xác định 6 ô xung quanh một quả bóng bất kỳ.

4.3.2 Cơ chế nhắm bắn và đạn bay

- Low-Pass Filter (Bộ lọc thông thấp):** Khi người chơi kéo thả ngắm bắn trên màn hình cảm ứng, dữ liệu thô (`aimX`, `aimY`) bị nhiễu và giật. Công thức

$$\text{smoothedAim} += (\text{aim} - \text{smoothedAim}) * \text{AIM_SMOOTH_FACTOR}$$

(với $\text{AIM_SMOOTH_FACTOR} = 0.3f$)

được sử dụng để làm mượt đường nét đứt, tạo chuyển động dự đoán quỹ đạo mềm mại.

- **Phản xạ góc bắn:** Khi đạn bay ($\text{bulletX} += vx$), nếu chạm biên trái (LEFT_WALL) hoặc biên phải (RIGHT_WALL), vận tốc trục x (vx) bị đảo dấu ($vx = -vx$) và tọa độ được dời lại để chống tình trạng bóng dính vào tường.
- **Tối ưu hóa kiểm tra va chạm (Collision Detection):** Việc tính khoảng cách thực tế giữa hai điểm yêu cầu hàm căn bậc hai $\text{sqrt}()$ – một phép toán rất tốn tài nguyên trên Cortex-M4. Để tối ưu, project đã bình phương hai vế và so sánh trực tiếp với hằng số COLLISION_DIST_SQ ($24^2 = 576$).

Trích dẫn *Model.cpp* (bên trong hàm *isCollisionAt*):

```
float dx = bulletX - cx;
float dy = bulletY - cy;
if (dx*dx + dy*dy <= COLLISION_DIST_SQ) { ... } // Không cần gọi sqrt()
```

Cách tiếp cận này tiết kiệm đáng kể chu kỳ CPU, đảm bảo frame rate luôn giữ ở mức 60 Hz ngay cả khi bóng bay sát lưới chứa nhiều bóng.

4.3.3 Thuật toán BFS xử lý nổ bóng và rút bóng

Sau khi đạn va chạm và ghim vào lưới (*snapToGrid*), hệ thống kích hoạt liên tiếp hai quá trình duyệt đồ thị (Graph Traversal) dựa trên thuật toán **BFS (Breadth-First Search)**:

1. **Tìm bóng cùng màu (Match Checking):** Bắt đầu từ vị trí bóng vừa bắn, BFS tỏa ra 6 hướng lân cận để gom tất cả các quả bóng cùng màu kết nối với nhau. Nếu tập hợp tìm được có kích thước ≥ 3 (Match), toàn bộ bóng trong tập hợp sẽ bị xóa (đổi thành *EMPTY_COLOR*), cộng điểm và phát âm thanh *audio_sfx_match*.
2. **Tìm bóng rơi (Floating Detection):** Khi một chùm bóng bị nổ, có thể làm rung các quả bóng khác đang bám vào chúng (chùm bóng bị đứt liên kết với trần nhà). Bài toán này giải bằng hàm *checkFloatingBalls*:
 - Chạy BFS bắt đầu từ tất cả các quả bóng ở hàng trên cùng (trần nhà).
 - Đánh dấu (visited) mọi quả bóng có thể đi tới được từ trần.
 - Duyệt lại toàn bộ lưới một lần nữa: bất kỳ quả bóng nào tồn tại nhưng không được đánh dấu nghĩa là nó đang lơ lửng. Chúng lập tức bị xóa (rơi xuống) và cộng điểm combo.

5. ĐÁNH GIÁ HIỆU NĂNG VÀ CÁC KHÓ KHĂN

5.1 Hiệu năng và Tài nguyên

5.1.1 Sử dụng bộ nhớ RAM

Hệ thống sử dụng đồng thời hai vùng nhớ: SRAM nội (256KB, tốc độ cao, gắn liền lõi CPU) và SDRAM ngoại (8MB, dung lượng lớn, kết nối qua bộ điều khiển FMC). Việc phân bổ dữ liệu vào đúng vùng nhớ là yếu tố quyết định giúp hệ thống chạy ổn định dù tài nguyên SRAM rất hạn chế.

Vùng SDRAM ngoại (8MB) – chứa khung hình đồ họa:

TouchGFX được cấu hình chạy chế độ Double Buffering, nghĩa là hệ thống luôn giữ 2 khung hình cùng lúc – một khung đang hiển thị lên LCD, một khung đang được vẽ ở hậu trường – để chuyển cảnh mượt mà, không giật hình.

Với độ phân giải màn hình 240×320 pixel, định dạng màu RGB565 (2 byte/pixel), mỗi khung hình chiếm:

$$240 \times 320 \times 2 \text{ byte} = 153.6 \text{ KB}$$

Do dùng 2 khung hình song song, tổng dung lượng cần thiết là:

$$153.6 \text{ KB} \times 2 = 307.2 \text{ KB}$$

Con số này vượt quá toàn bộ 256KB SRAM nội của vi điều khiển – vì vậy 2 khung hình bắt buộc phải đặt ở SDRAM ngoại. Ta có thể thấy trong mã nguồn: file linker script STM32F429ZITX_FLASH.ld (dòng 68) khai báo vùng nhớ SDRAM bắt đầu tại địa chỉ 0xD0000000 (thuộc FMC Bank 5 – nơi kết nối chip SDRAM 8MB trên board), và TouchGFXGeneratedHAL.cpp đặt framebuffer vào đúng vùng nhớ này thông qua section "TouchGFX_Framebuffer".

Vùng SRAM nội (256KB) — chứa dữ liệu xử lý thời gian thực:

Vì khung hình đồ họa đã được chuyển hết ra SDRAM, SRAM nội chỉ còn phải phục vụ các tác vụ nhỏ, cần truy xuất nhanh:

Thành phần	Dung lượng	Ghi chú
Bộ đệm âm thanh (audio_buffer)	8.19 KB	Phục vụ truyền DMA cho I2S
Mảng lưới bóng (grid)	90 byte	$MAX_ROWS \times MAX_COLS = 10 \times 9$
Stack, Heap, biến cục bộ khác	Phần còn lại	Dự phòng cho vòng lặp game và TouchGFX runtime

Với tổng nhu cầu chỉ vài chục KB trong khi SRAM có sẵn 256KB, hệ thống còn rất nhiều dư địa an toàn — không có rủi ro tràn bộ nhớ ở vùng này.

5.1.2 Sử dụng bộ nhớ Flash

Toàn bộ dữ liệu âm thanh (nhạc nền dạng ADPCM nén và hiệu ứng SFX dạng PCM thô) được lưu cố định trong Flash nội 2MB. Nhờ áp dụng nén ADPCM cho nhạc nền – giảm dung lượng khoảng 4 lần so với PCM thô – tổng dung lượng âm thanh chỉ chiếm khoảng 1.15MB, giúp toàn bộ chương trình (bao gồm cả code) nằm gọn trong giới hạn 2MB mà không xảy ra lỗi tràn bộ nhớ khi biên dịch.

5.1.3 Độ trễ xử lý input

Nút bấm được quét theo chu kỳ khung hình 60Hz, tương ứng độ trễ tối đa từ lúc nhấn đến lúc xử lý là 1/60 giây (~16.6ms). So với ngưỡng cảm nhận độ trễ của con người trong các thao tác thông thường (khoảng 100ms), con số này là rất tốt và không gây cảm giác trễ khi chơi.

5.2 Khó khăn trong project

5.2.1 Vấn đề 1: Lỗi Linker "Undefined reference"

- **Dấu hiệu:** IDE báo lỗi Linker khi gọi hàm `AudioManager::playBGM()`.
- **Nguyên nhân kỹ thuật:** Tập `AudioManager.cpp` được sinh ra sau khi tạo C++ Project, nên file `.cproject` chưa tự động lập chỉ mục Index. Trình biên dịch gcc bỏ qua việc biên dịch tập này ra tập `.o`.
- **Khắc phục:** Thực hiện Refresh Project Tree trên IDE hoặc chọn tập → Properties → C/C++ Build → Bỏ tick "Exclude from build".

5.2.2 Vấn đề 2: Nguy cơ tràn SRAM nội do Double Buffering

- **Dấu hiệu:** Về lý thuyết, nếu framebuffer đặt trong SRAM nội, dự án sẽ không thể build được do vượt quá dung lượng bộ nhớ cho phép.
- **Nguyên nhân kỹ thuật:** TouchGFX được cấu hình chạy chế độ Double Buffering, mỗi khung hình RGB565 240×320 chiếm 153.6KB, tổng 2 khung là 307.2KB – vượt quá toàn bộ 256KB SRAM nội của MCU.
- **Khắc phục:** Nhóm đã cấu hình đưa framebuffer ra vùng SDRAM ngoại 8MB thông qua bộ điều khiển FMC (địa chỉ 0xD0000000), giải quyết triệt để giới hạn bộ nhớ trong khi vẫn giữ được hiệu ứng chuyển cảnh mượt mà của Double Buffering.

5.2.3 Vấn đề 3: Đánh đổi giữa giải nén ADPCM thời gian thực và tải CPU

- **Dấu hiệu:** Nhạc nền BGM cần giải mã ADPCM ngay trong callback `HAL_I2S_TxCpltCallback/HAL_I2S_TxCpltCallback` – đây là ngữ cảnh ngắt (interrupt context), thời gian xử lý bị giới hạn nghiêm ngặt.
- **Nguyên nhân kỹ thuật:** Nếu thời gian giải mã một nửa buffer lâu hơn thời gian phát nửa buffer đó qua I2S, hệ thống sẽ xảy ra hiện tượng "audio underrun" (âm thanh bị khựng/rè do DMA không kịp nạp dữ liệu mới).
- **Hướng xử lý/lưu ý:** Nhóm chưa đo đạc định lượng thời gian giải mã thực tế (do thiếu công cụ profiling — xem thêm ở mục hạn chế 6.2), nên đây là điểm cần được kiểm chứng kỹ hơn nếu mở rộng thêm bản nhạc nền có nhịp độ phức tạp hơn.

5.2.4 Vấn đề 4: Thiếu cơ chế chống dội phím (debounce) phần cứng/phần mềm

- **Dấu hiệu:** Với nút bấm cơ học đã qua sử dụng nhiều lần, tiếp điểm có thể rung ở mức điện áp trong vài mili-giây khi nhấn/nhả, dẫn đến khả năng 1 lần bấm bị đọc thành 2-3 lần.
- **Nguyên nhân kỹ thuật:** project chỉ dùng kỹ thuật bắt sườn xung đơn giản (`if (isShoot && !prevShoot)`), không có bộ lọc thời gian hay đếm mẫu ổn định.
- **Hướng xử lý:** Đây là hạn chế được nhóm ghi nhận nhưng chưa khắc phục trong phạm vi đồ án này (xem thêm mục 6.2 – Hướng phát triển).

6. KẾT LUẬN

6.1 Tổng kết kết quả đạt được

Qua quá trình thực hiện project, nhóm đã xây dựng thành công trò chơi Dynamite trên nền tảng STM32F429I Discovery, đáp ứng đầy đủ các mục tiêu đề ra ban đầu về cả phần cứng lẫn phần mềm. Cụ thể:

- Về kiến trúc phần mềm: Project áp dụng thành công mô hình Model-View-Presenter (MVP) của TouchGFX, tách biệt rõ ràng giữa logic xử lý game (Model) và giao diện hiển thị (View/Presenter), giúp mã nguồn dễ bảo trì và mở rộng.
- Về xử lý đồ họa: Nhóm đã khai thác hiệu quả bộ điều khiển FMC để đưa framebuffer ra vùng SDRAM ngoại (địa chỉ 0xD0000000), cho phép sử dụng chiến lược Double Buffering mà không bị giới hạn bởi dung lượng SRAM nội hạn chế (256KB) của vi điều khiển.
- Về xử lý âm thanh: Việc kết hợp DMA truyền vòng lặp (Circular Mode) cho ngoại vi I2S với thuật toán giải mã ADPCM thời gian thực đã giải quyết đồng thời hai bài toán: giải phóng CPU khỏi tác vụ phát âm thanh (không gây gián đoạn khi TouchGFX đang render đồ họa), đồng thời tiết kiệm đáng kể dung lượng Flash để toàn bộ tài nguyên âm thanh có thể nằm gọn trong 2MB bộ nhớ chương trình.
- Về xử lý input: Hệ thống 4 nút bấm vật lý kết hợp cảm ứng màn hình hoạt động ổn định, đáp ứng đủ nhanh (độ trễ lý thuyết ~16.6ms) cho trải nghiệm chơi game thời gian thực.

Kết quả cho thấy nhóm đã nắm vững và vận dụng tốt các kiến thức về lập trình nhúng, xử lý ngoại vi (GPIO, I2S, DMA, FMC/SDRAM) và tư duy thiết kế phần mềm hướng đối tượng trên hệ thống tài nguyên hạn chế.

6.2 Hạn chế và hướng phát triển

Bên cạnh các kết quả đạt được, project vẫn còn một số hạn chế mà nhóm nhận thấy cần cải thiện trong tương lai:

- Thiếu cơ chế chống dội phím (debounce) chuyên dụng: Hiện tại hệ thống chỉ dừng ở mức phát hiện sườn xung đơn giản, chưa có bộ lọc chống nhiễu cơ học. Hướng phát triển là bổ sung debounce bằng phần mềm (đếm mẫu ổn định qua timer) hoặc phần cứng (mạch lọc RC).
- Chưa có công cụ đo lường hiệu năng (profiling): Project chưa tích hợp cơ chế đo tải CPU thời gian thực bằng thanh ghi DWT->CYCCNT. Việc bổ sung module này sẽ giúp đánh giá định lượng chính xác hơn về hiệu năng hệ thống khi vừa xử lý đồ họa vừa giải mã âm thanh.

- Khả năng mở rộng nội dung game: Trong tương lai, có thể bổ sung thêm các chế độ chơi mới, hệ thống lưu điểm cao (high score) sử dụng bộ nhớ Flash còn trống, hoặc mở rộng thư viện âm thanh với nhiều hiệu ứng đa dạng hơn.

6.3 Lời cảm ơn

Nhóm thực hiện xin gửi lời cảm ơn chân thành đến Thầy Ngô Lam Trung đã tận tình hướng dẫn, góp ý và tạo điều kiện thuận lợi trong suốt môn học. Nhóm cũng xin cảm ơn Nhóm chuyên môn Hệ nhúng đã cung cấp trang thiết bị và môi trường học tập cần thiết để nhóm có thể hoàn thành sản phẩm này.

Trong quá trình thực hiện, do kiến thức và thời gian còn hạn chế, project khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý từ quý thầy/cô để hoàn thiện hơn trong tương lai.